

week-3-day-2: user-stories-testcase generator

1. User story -tests generator -- with vibe coding implement the Jira integration to fetch user stories (both frontend and backend)

The image shows two screenshots of a web application running on localhost:5173. The browser's tab bar includes 'User Story', 'GitHub Au', 'Copilot', 'notebook', 'Google N', 'Notebook', '[US-25] E', 'JIRA Acces', and 'New tab'.

Top Screenshot: User Story to Tests

The page has a title 'User Story to Tests' and a subtitle 'Generate comprehensive test cases from your user stories'. A dark blue button labeled 'Connect to JIRA' is centered in a white box. Below this, there are two input fields: 'Story Title *' with a placeholder 'Enter the user story title...' and 'Description' with a placeholder 'Additional description (optional)...'.

Bottom Screenshot: JIRA Connection Settings

This screen shows a dark blue button labeled 'JIRA Connection Settings'. Below it, there are three input fields: 'JIRA End Point' with the value 'https://sowmyanarayanan.atlassian.net', 'JIRA User ID' with the value 'sowmyanarayanan.sailapathi@gmail.com', and 'JIRA API KEY' with a masked value '.....'. A dark blue 'Connect' button is below these fields. At the bottom, there is a 'User Story ID' input field with a placeholder 'Enter the JIRA User Story ID'.

User Story ID

US-25

Load Story

Connected to JIRA successfully.

Story Title *

Enter the user story title...

Description

Additional description (optional)...

Acceptance Criteria *

Story Title *

Employee Profile Employee Self

Description

As a user, I want to employee profile employee self so that the related functionality works as expected.

Acceptance Criteria *

Given the system is operational, when the user performs the intended action, then the system should respond accurately and meet functional expectations.

Additional Info

Any additional information (optional)...

Generated Test Cases

10 test case(s) generated • Model: openai/gpt-oss-120b • Tokens: 2106

Test Case ID	Title	Category	Priority	Expected Result
TC-001	View own employee profile successfully	Positive	P1	The employee's profile information is displayed correctly, matching the data stored in the system
TC-002	Edit employee profile with valid data and save	Positive	P1	A success message is shown and the updated information is displayed on the profile page
TC-003	Attempt to save profile with invalid email format	Negative	P2	An inline validation error is displayed indicating the email format is invalid and the profile is not saved
TC-004	Attempt to save profile with a required field left empty	Negative	P2	A validation error is shown for the empty required field and the profile is not saved

2. Application is User story - tests generator, Add a button option to Generate Feature file for the user stories

Story Title *

Employee Profile Employee Self

Description

As a user, I want to employee profile employee self so that the related functionality works as expected.

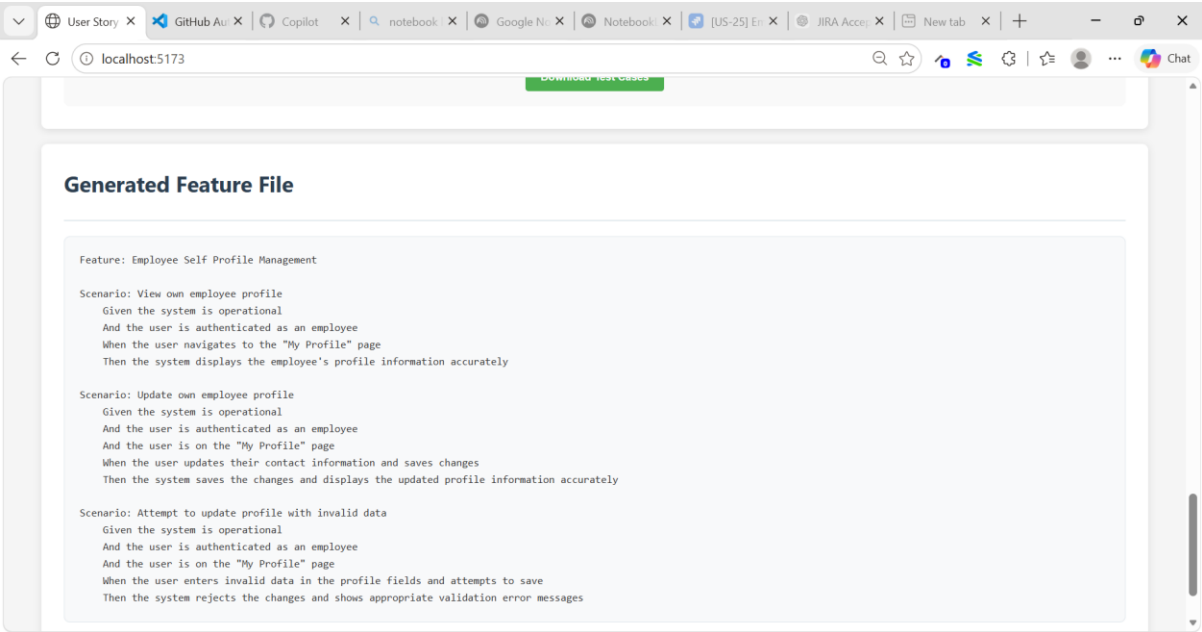
Acceptance Criteria *

Given the system is operational, when the user performs the intended action, then the system should respond accurately and meet functional expectations.

Additional Info

Any additional information (optional)...

Generate Test Case Generate Feature File



Additional Features:

A. Include Priority Column for Test Cases:

Test Case ID	Title	Category	Priority	Expected Result
TC-001	View own employee profile successfully	Positive	P1	The employee's profile information is displayed correctly, matching the data stored in the system
TC-002	Edit employee profile with valid data and save	Positive	P1	A success message is shown and the updated information is displayed on the profile page
TC-003	Attempt to save profile with invalid email format	Negative	P2	An inline validation error is displayed indicating the email format is invalid and the profile is not saved
TC-004	Attempt to save profile with a required field left empty	Negative	P2	A validation error is shown for the empty required field and the profile is not saved
TC-005	Upload profile picture at maximum allowed size	Edge	P3	The picture is uploaded successfully, displayed on the profile, and a success notification appears
TC-006	Upload profile picture exceeding size limit	Negative	P3	An error message is displayed indicating the file exceeds the maximum size and the upload is rejected
TC-007	Prevent access to another employee's profile via URL manipulation	Authorization	P1	The system returns an authorization error or redirects to Employee A's own profile, preventing access to Employee B's data

B. Download Test Cases in .xlsx and .csv format:

User Story xGitHub Au xCopilot xnotebook xGoogle N xNotebook x[US-25] E xJIRA AccexNew tab x+--oX

localhost:5173

Chat

TC-005	Upload profile picture at maximum allowed size	Edge	P3	The picture is uploaded successfully, displayed on the profile, and a success notification appears
TC-006	Upload profile picture exceeding size limit	Negative	P3	An error message is displayed indicating the file exceeds the maximum size and the upload is rejected
TC-007	Prevent access to another employee's profile via URL manipulation	Authorization	P1	The system returns an authorization error or redirects to Employee A's own profile, preventing access to Employee B's data
TC-008	Unauthenticated user attempts to access the profile page	Authorization	P1	The user is redirected to the login page with a message indicating authentication is required
TC-009	Profile page loads within acceptable performance threshold	Non-Functional	P2	The profile page fully renders in 2 seconds or less under normal load conditions
TC-010	Profile changes persist after logout and subsequent login	Positive	P2	The previously saved changes are displayed, confirming data persistence

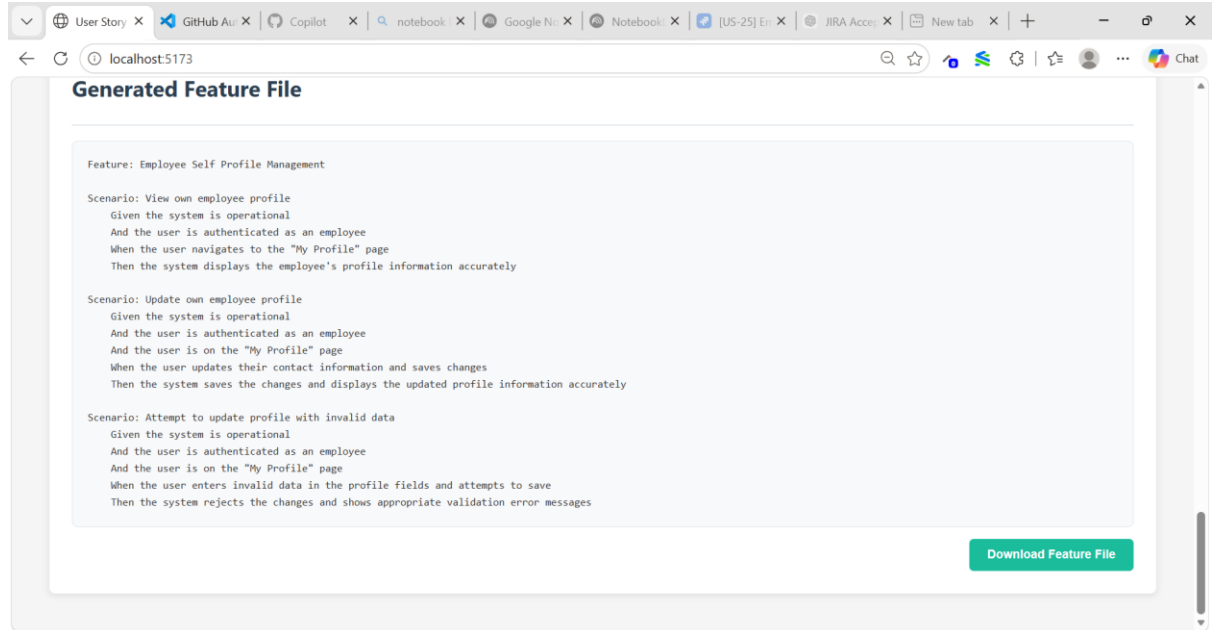
☒ Excel (.xlsx) ☐ CSV (.csv)

Download Test Cases



test-cases.csv

C. Download Feature File with extension. Feature



The screenshot shows a web browser window with multiple tabs. The active tab is titled 'localhost:5173'. The page content is titled 'Generated Feature File' and displays a Gherkin feature file for 'Employee Self Profile Management'. The feature file includes three scenarios: 'View own employee profile', 'Update own employee profile', and 'Attempt to update profile with invalid data'. Each scenario is written in Gherkin syntax with 'Given', 'And', 'When', and 'Then' clauses. A green button labeled 'Download Feature File' is located at the bottom right of the feature file content area.

```
Feature: Employee Self Profile Management

Scenario: View own employee profile
  Given the system is operational
  And the user is authenticated as an employee
  When the user navigates to the "My Profile" page
  Then the system displays the employee's profile information accurately

Scenario: Update own employee profile
  Given the system is operational
  And the user is authenticated as an employee
  And the user is on the "My Profile" page
  When the user updates their contact information and saves changes
  Then the system saves the changes and displays the updated profile information accurately

Scenario: Attempt to update profile with invalid data
  Given the system is operational
  And the user is authenticated as an employee
  And the user is on the "My Profile" page
  When the user enters invalid data in the profile fields and attempts to save
  Then the system rejects the changes and shows appropriate validation error messages
```

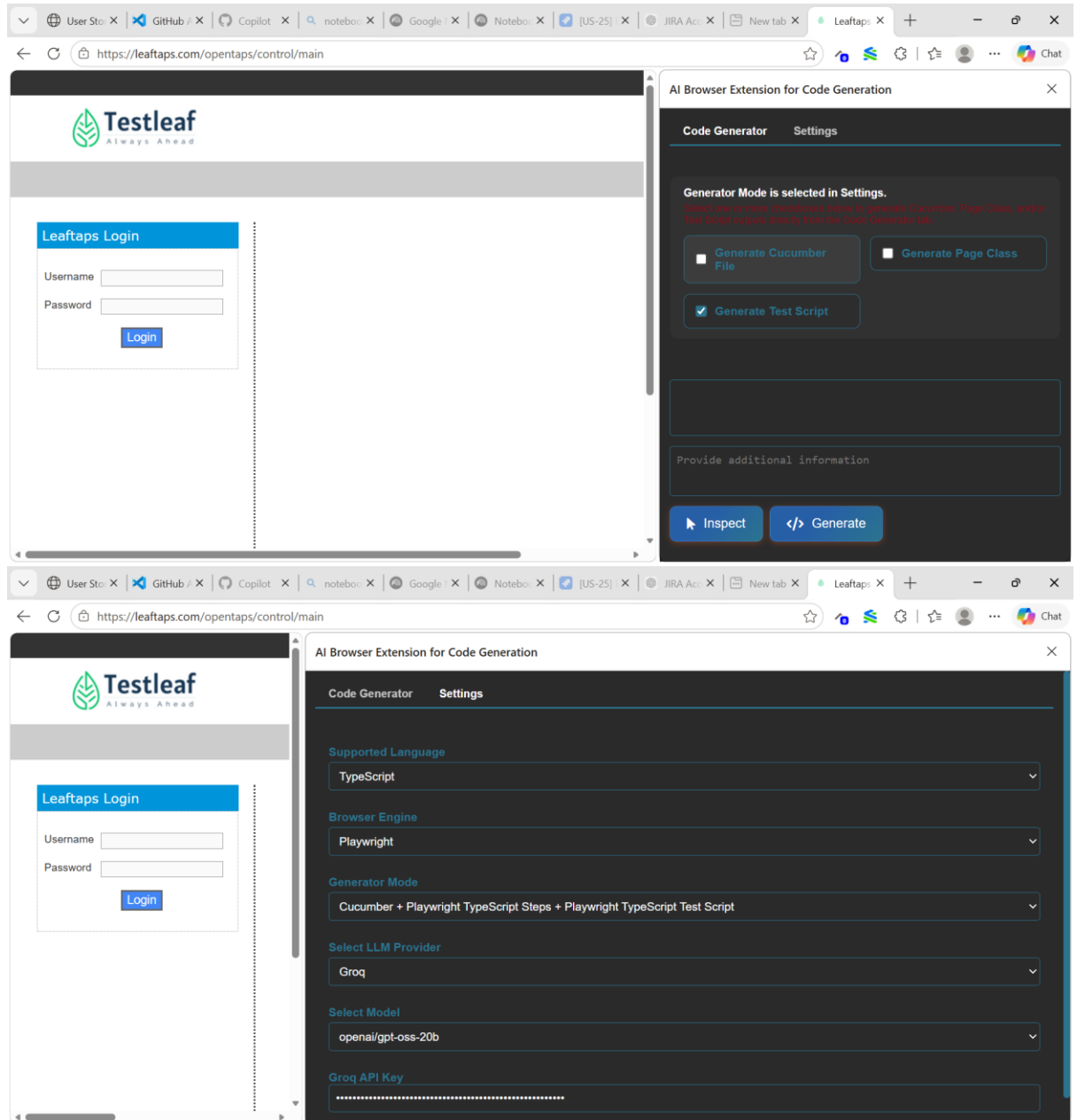
Download Feature File



Employee-Profile-Em
ployee-Self.feature

3. AI Browser Extension - With vibe coding generate the test scripts in the language selected (tool/language of your choice)

Playwright + Type Script:



Feature File:

The screenshot shows the LeafTaps application on the left and the AI Browser Extension for Code Generation on the right. The application has a 'LeafTaps Login' form with 'Username' and 'Password' fields and a 'Login' button. The extension interface has tabs for 'Code Generator' and 'Settings'. Under 'Code Generator', there are three checked options: 'Generate Cucumber File', 'Generate Page Class', and 'Generate Test Script'. Below these, there is a 'Reset Chat' button and a chat area containing a feature file for 'Login functionality for OpenTaps'.

Feature: Login functionality for OpenTaps

Scenario Outline: Successful login with valid credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should see the dashboard page

Examples:

username	password
arun.kumar	Arun@1234

Page Class:

The screenshot shows the LeafTaps application on the left and the AI Browser Extension for Code Generation on the right. The application is the same as in the previous screenshot. The extension interface is the same, but the chat area now displays a Page Class for the login form.

```
import { Page, Locator } from '@playwright/test';

/**
 * Page Object representing the login form on the OpenTaps application.
 *
 * @remarks
 * This class encapsulates all interactions with the login form, providing
 * clear, typed methods for automation engineers to use in their tests.
 */
export class LoginPage {
  readonly page: Page;
  readonly usernameInput: Locator;
```


Test Script:

The screenshot shows a web browser with the URL `https://leafTaps.com/opentaps/control/main`. The main content area displays the "LeafTaps Login" page, which includes a "Username" input field, a "Password" input field, and a "Login" button. The page header features the "Testleaf Always Ahead" logo.

On the right side, an "AI Browser Extension for Code Generation" sidebar is open. It contains a "Reset Chat" button and a code editor with the following Playwright test script:

```
import { test, expect } from '@playwright/test';

test.describe('Login Flow', () => {
  // Base URL for all tests
  test.use({ baseURL: 'https://leafTaps.com' });

  // Navigate to the login page before each test
  test.beforeEach(async ({ page }) => {
    await page.goto('/opentaps/control/main');
  });

  test('should log in successfully with valid credentials', async ({ page }) => {
    // Fill in the username and password fields
    await page.fill('input[name="USERNAME"]', 'validUser');
    await page.fill('input[name="PASSWORD"]', 'validPass');
  });
});
```

Below the code editor is a text input field labeled "Provide additional information" and two buttons: "Inspect" and "Generate".

Selenium + Python:

The screenshot shows the LeafTaps web application in a browser. The main content area displays the 'LeafTaps Login' form with fields for 'Username' and 'Password', and a 'Login' button. The 'AI Browser Extension for Code Generation' sidebar is open on the right, showing the 'Settings' tab. The settings include:

- Supported Language: Python
- Browser Engine: Selenium
- Generator Mode: Cucumber + Selenium Python Steps + Selenium
- Select LLM Provider: Groq
- Select Model: openai/gpt-oss-20b
- Groq API Key: [Redacted]

Feature File:

The screenshot shows the LeafTaps web application in a browser. The main content area displays the 'LeafTaps Login' form. The 'AI Browser Extension for Code Generation' sidebar is open on the right, showing the 'Generate Test Script' tab. The sidebar contains a 'Reset Chat' button and a text area with the following content:

Feature: Login to OpenTaps

Scenario Outline: Successful login with valid credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should be logged in successfully

Examples:

username	password
kumar123	passKumar1
rani456	raniPass@2
sarika789	Sarika#3

Provide additional information

Page Class:

The screenshot displays a web browser window with the URL `https://leafTaps.com/opentaps/control/main`. The main content area shows the "LeafTaps Login" page, which includes a "Username" input field, a "Password" input field, and a "Login" button. The page header features the "Testleaf Always Ahead" logo.

On the right side, an "AI Browser Extension for Code Generation" sidebar is open. It contains three tabs: "Generate Cucumber File", "Generate Page Class", and "Generate Test Script". The "Generate Page Class" tab is selected. The sidebar also includes a "Reset Chat" button and a text input field labeled "Provide additional information".

The code generated in the sidebar is as follows:

```
from selenium.webdriver.common.by import By
from selenium.webdriver.remote.webdriver import WebDriver
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException

class LoginPage:
    """
    Page Object representing the login form.

    Provides methods to interact with the username and password fields
    and to submit the form. Explicit waits are used to ensure elements
    are ready before any action is performed.
    """
```

The second screenshot shows the same browser window, but the code generated in the sidebar is different:

```
def __init__(self, driver: WebDriver, timeout: int = 10) -> None:
    """
    Initialize the LoginPage with a WebDriver instance.

    :param driver: Selenium WebDriver instance.
    :param timeout: Maximum wait time for element visibility or clickability.
    """
    self.driver = driver
    self.wait = WebDriverWait(driver, timeout)

def _username_field(self):
    """Return the username input element after ensuring it is visible."""
    return self.wait.until(
        EC.visibility_of_element_located((By.ID, "username"))
    )
```

Test Script:

The screenshot displays a web browser window with the Testleaf login page on the left and an AI Browser Extension for Code Generation sidebar on the right. The browser's address bar shows the URL <https://leaf taps.com/opentaps/control/main>. The Testleaf logo, featuring a green leaf icon and the text "Testleaf Always Ahead", is at the top of the page. Below the logo is a "Leaf taps Login" section with input fields for "Username" and "Password", and a "Login" button. The AI Browser Extension sidebar, titled "AI Browser Extension for Code Generation", has three tabs: "Generate Cucumber File", "Generate Page Class", and "Generate Test Script". The "Generate Test Script" tab is active, showing a "Reset Chat" button and a code editor with the following Python code:

```
import pytest
from selenium import webdriver
from selenium.webdriver.chrome.service import Service as ChromeService
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager

@pytest.fixture(scope="session")
def driver():
    """
    Pytest fixture that sets up a Chrome browser instance,
    yields it to the test, and ensures cleanup afterwards.
    """
    options = webdriver.ChromeOptions()
    options.add_argument("--headless")  # Run headless for CI environments

    Provide additional information
```

The browser's taskbar at the bottom shows several open tabs: "User Sto...", "GitHub", "Copilot", "notebo...", "Google", "Notebo...", "[US-25]", "JIRA Ac...", "New tab", "Leaf taps", and "Chat".

